

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

Task Management REST API

Detailed project brief generated from the project record.

DURATION

14 days

TECHNOLOGY STACK

Node.js

Express.js

MongoDB

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

Build a small Node.js/Express service with MongoDB that lets users register, log in, and manage personal to do tasks via a RESTful API.

TYPE

backend

DURATION

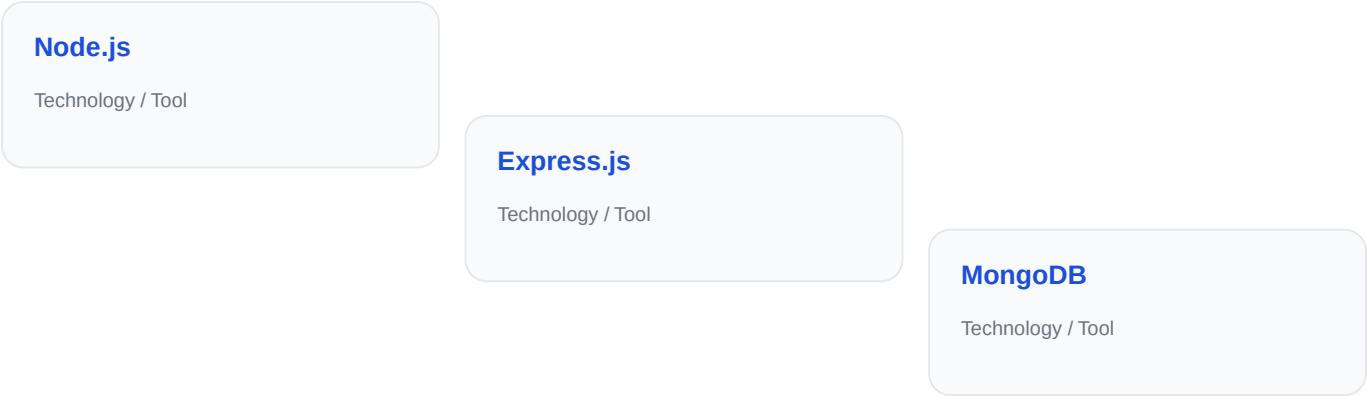
14 days

Primary Objectives

- Implement JWT based authentication.
- Create CRUD endpoints for tasks.
- Validate request payloads and return proper HTTP status codes.
- Write unit and integration tests covering core routes.
- Document API endpoints with OpenAPI/Swagger.

Technology Stack

Technologies, frameworks and tools expected for implementation.



Functional Requirements

- Users can register with email and password.
- Registered users can log in and receive a JWT.
- Authenticated users can create a task (title, description, dueDate).
- Authenticated users can retrieve a paginated list of their tasks.
- Authenticated users can update or delete their own tasks.

- The API returns appropriate validation errors for malformed requests.

Non-Functional Requirements

- Passwords must be hashed with bcrypt.
- All endpoints must respond within 200 ms for typical payloads.
- Input data must be validated using a schema library.
- Error responses must not expose stack traces.
- API documentation must be generated automatically.

Modules & Features

Major functional areas that make up the project.

User Auth

Handles registration, login, and JWT issuance.

- Register endpoint
- Login endpoint
- Password hashing
- Token generation

Task Management

CRUD operations for personal tasks.

- Create task
- Read tasks (list & detail)
- Update task
- Delete task

Validation & Middleware

Centralized request validation and auth guard.

- Schema validation
- JWT verification middleware
- Error formatting middleware

Persistence Layer

MongoDB models and data access helpers.

- User schema
- Task schema
- Repository functions

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Project Setup

Create repo with npm init, Add Express, Mongoose, dotenv, Write connection module, Add /health route

- Create repo with npm init
- Add Express, Mongoose, dotenv
- Write connection module
- Add /health route

2

Authentication

Define User model, Add bcrypt hashing, Create register & login routes, Generate JWT on login

- Define User model
- Add bcrypt hashing
- Create register & login routes
- Generate JWT on login

3

Task CRUD

Define Task model, Create router with POST, GET, PATCH, DELETE, Apply auth middleware

- Define Task model
- Create router with POST, GET, PATCH, DELETE
- Apply auth middleware

4

Validation & Error Handling

Integrate Joi (or celebrate) schemas, Add validation middleware, Implement global error handler

- Integrate Joi (or celebrate) schemas
- Add validation middleware
- Implement global error handler

5

Testing & Documentation

Write unit tests with Jest, Write integration tests with Supertest, Add Swagger definition and UI route

- Write unit tests with Jest
- Write integration tests with Supertest
- Add Swagger definition and UI route

API Design

Key API endpoints and expected responsibilities.

POST

/auth/register

Create new user

POST

/auth/login

Authenticate and receive JWT

POST

/tasks

Create a new task

GET

/tasks

List user's tasks with pagination

GET

/tasks/:id

Get task details

PATCH

/tasks/:id

Update task fields

DELETE**/tasks/:id**

Delete a task

Evaluation Criteria

Criteria that can be used to assess the completed project.

- Criterion: Correctness | Description: All functional requirements work as specified. | Weight: 30
- Criterion: Code Quality | Description: Readable, modular code with proper naming. | Weight: 15
- Criterion: Security | Description: Passwords hashed, JWT verified, no sensitive data leaks. | Weight: 15
- Criterion: Testing | Description: Automated tests cover core paths and edge cases. | Weight: 15
- Criterion: Documentation | Description: README and Swagger docs are clear and complete. | Weight: 15
- Criterion: Error Handling | Description: Consistent error responses and no uncaught exceptions. | Weight: 10

Expected Deliverables

Artifacts and outcomes expected at project completion.

- Complete source repository (Git).
- README with installation, run, and test instructions.
- Swagger/OpenAPI documentation.
- Postman collection (optional).
- Test suite with coverage report.
- Deployment script (e.g., Dockerfile).

Candidate Alignment

Project requirements compared with the candidate information available in the project record.

The project uses Node.js, Express, and MongoDB—exactly the technologies listed. It focuses on API development, matching the Backend Developer role and junior difficulty, while providing a realistic 14 day MVP scope.

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.